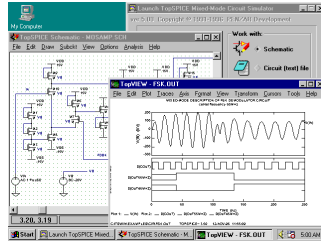


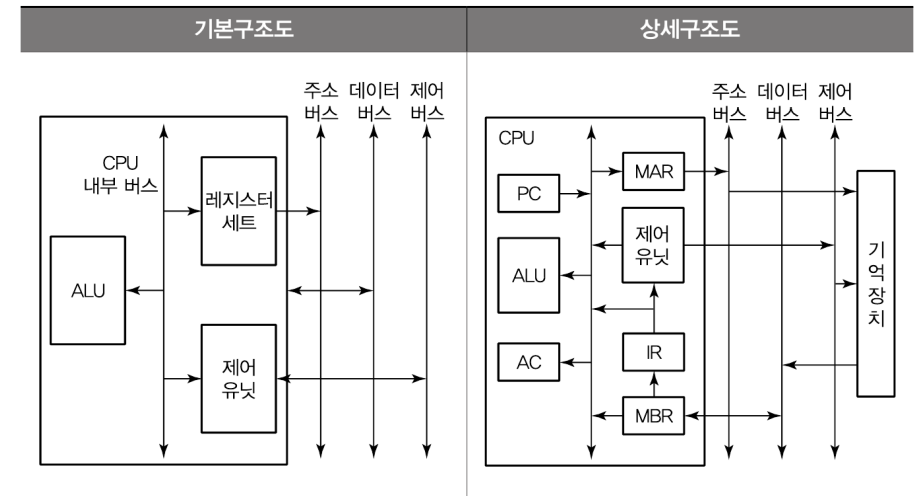
컴퓨터 시스템



CPU Internal Architecture

This material is reproduced based on
핵심 컴퓨터구조 및 운영체제 해설(최종언저, 예문사),
Computer Organization and Architecture, 9ed(by William
Stallings) and
Computer Architecture(by Behrooz Parhami),
Copyrights are reserved by corresponding authors.

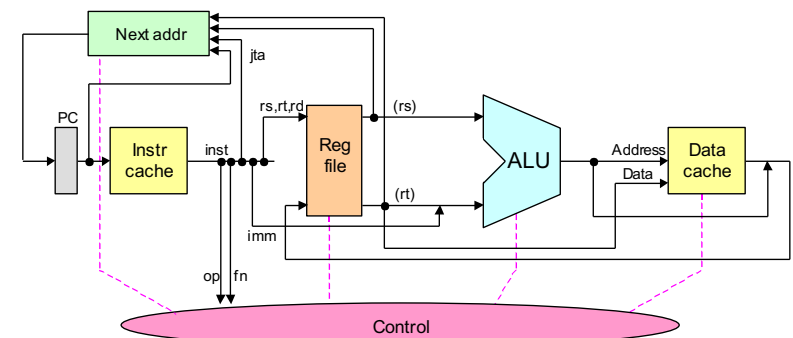
CPU 구조도



CPU 구성요소

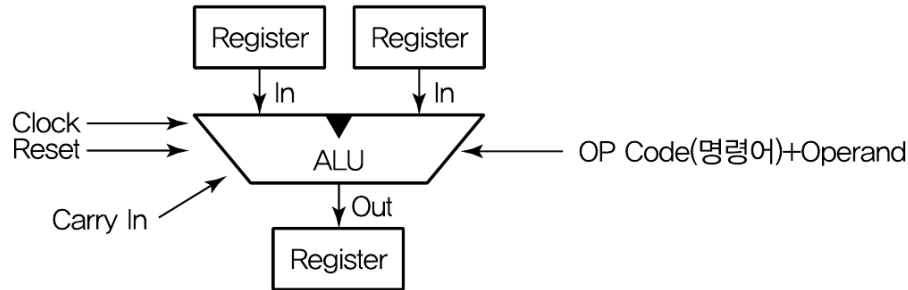
구성요소	설명
산술연산장치 (ALU)	각종 산술 연산들과 논리 연산들을 수행하는 회로들로 이루어진 하드웨어 모듈 산술 연산 : +, -, ×, ÷ 논리 연산 : AND, OR, NOT, XOR 등
제어 유닛 (CU)	프로그램 코드(명령어)를 해석하고, 그것을 실행하기 위한 제어 신호들(control signals)을 순차적으로 발생하는 하드웨어 모듈
레지스터 세트	액세스 속도가 가장 빠른 기억장치 CPU 내부에 포함할 수 있는 레지스터들의 수가 제한됨 (특수 목적용 레지스터들과 적은 수의 일반 목적용 레지스터들)
CPU 내부 버스	ALU와 레지스터들 간의 데이터 이동을 위한 데이터 선들과 제어 유닛으로부터 발생하는 제어 신호 선들로 구성된 내부 버스 외부의 시스템 버스들과는 직접 연결되지 않으며, 반드시 버퍼 레지스터들 혹은 시스템 버스 인터페이스 회로를 통하여 시스템 버스와 접속

The Instruction Execution Unit

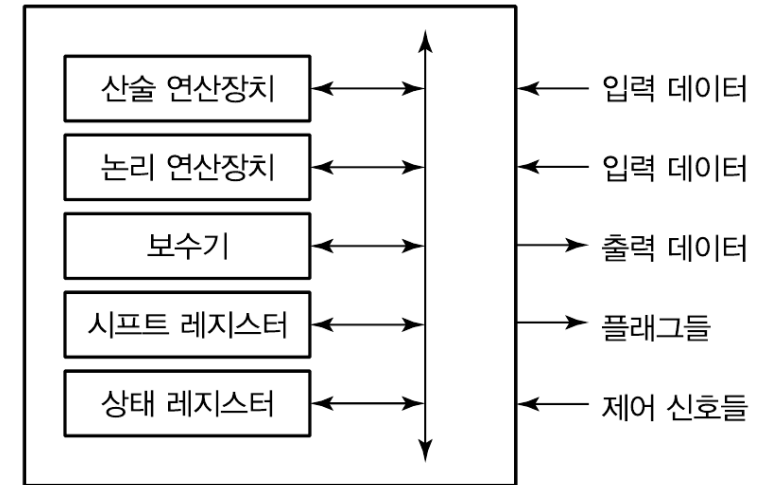


Abstract view of the instruction execution unit for MicroMIPS.

ALU(Arithmetic and Logic Unit)

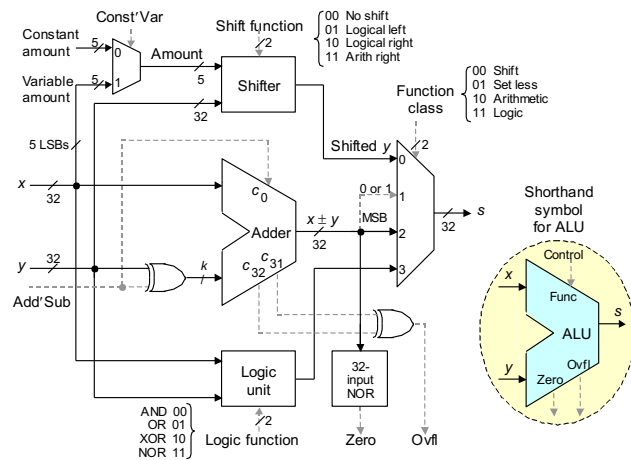


© 2021 Computer System



© 2021 Computer System

ALU Sample



© 2021 Computer System

Instruction Set

종류	설명
데이터 전송	레지스터와 레지스터 간, 레지스터와 기억장치 간, 혹은 기억장치와 기억장치 간에 데이터를 이동하는 동작
산술 연산	덧셈, 뺄셈, 곱셈 및 나눗셈과 같은 기본적인 산술 연산들
논리 연산	데이터의 각 비트들 간의 AND, OR, NOT 및 exclusive-OR 연산
입출력(I/O)	CPU와 외부 장치들 간의 데이터 이동을 위한 동작들
프로그램 제어	명령어 실행 순서를 변경하는 연산들 분기(branch), 서브루틴 호출(subroutine call)

© 2021 Computer System

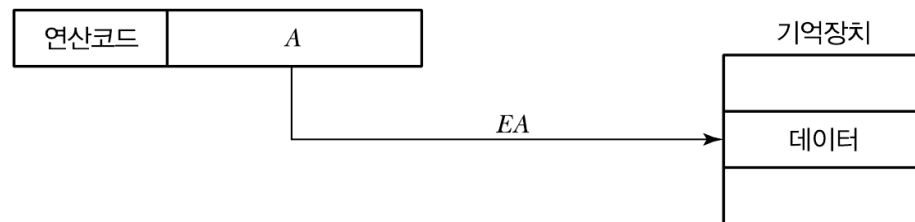
Instruction Set

- Register Type
 - 연산이 Register의 데이터에서만 이루어 지는 명령어
- Data Load from Memory
 - 연산에 필요한 데이터가 외부 메모리에 존재하여 메모리로부터 데이터를 읽는 동작이 필요한 명령어
- Data Save to Memory
 - 연산 결과를 외부 메모리에 저장하는 명령어
- Jump/Branch
 - 프로그램의 진행 순서를 변경하는 명령어

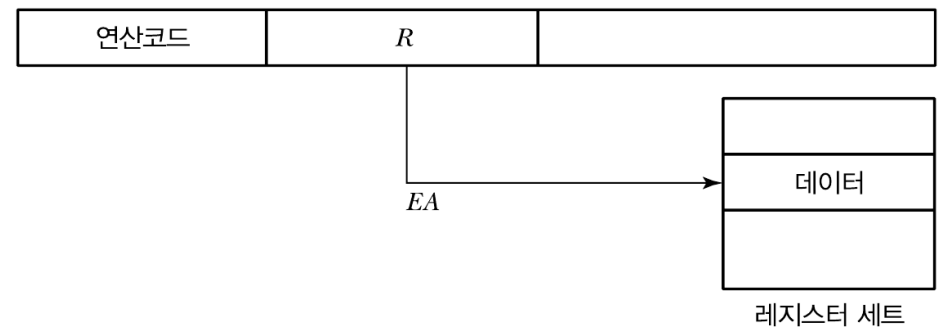
Addressing Mode

종류	설명	방식
직접 주소지정 방식	데이터가 저장된 기억장치의 위치를 지정	기억장치 주소
간접 주소지정 방식		
레지스터 주소지정 방식	데이터가 저장된 레지스터를 지정	레지스터 번호
레지스터 간접 주소지정 방식		
변위 주소지정 방식		
묵시적 주소지정 방식	명령어의 오퍼랜드 필드에 데이터가 포함	데이터
즉치 주소지정 방식		

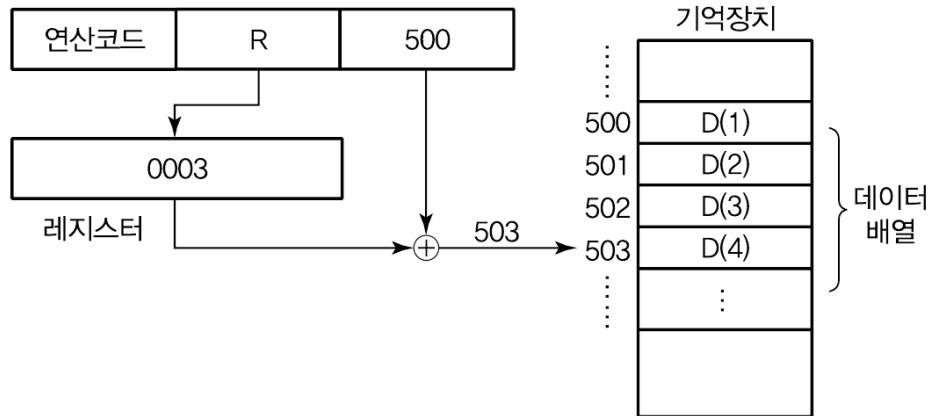
Direct Addressing



Register Addressing

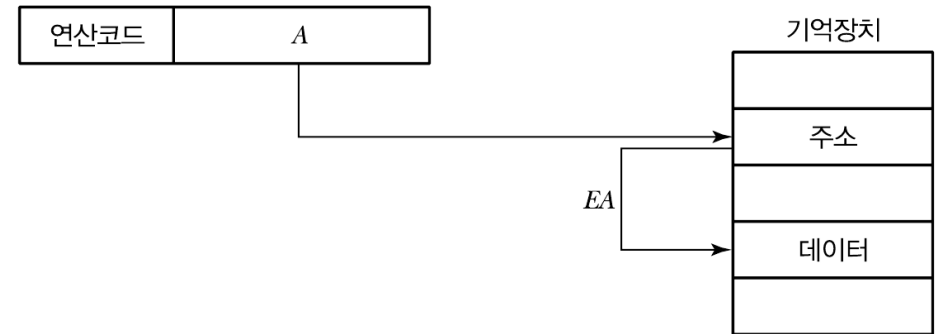


Register Indirect Addressing



© 2021 Computer System

Memory Indirect Addressing



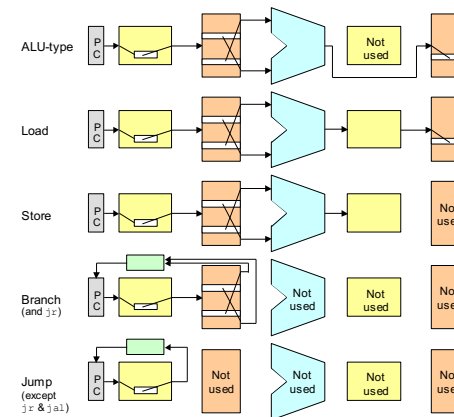
© 2021 Computer System

Immediate Addressing



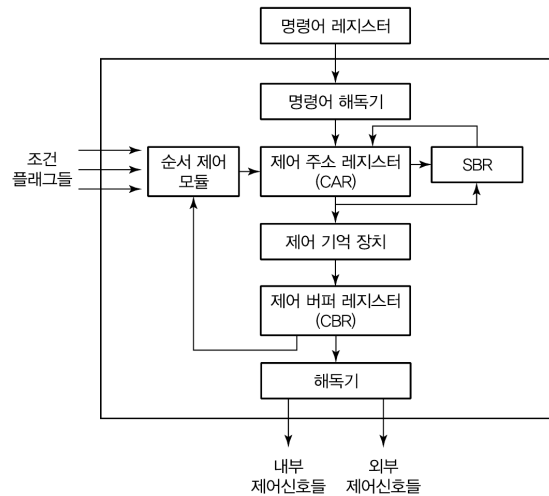
© 2021 Computer System

Performance of the Single-Cycle Design



© 2021 Computer System

CPU Control Unit



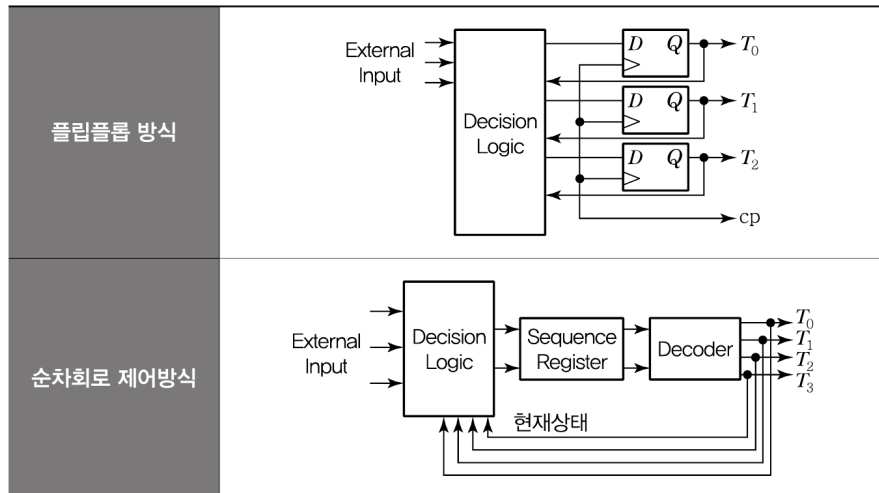
© 2021 Computer System

Control Unit 구성 요소

구성요소	실행기능
명령어 해독기 (instruction decoder)	명령어 레지스터(IR)로부터 들어오는 명령어의 연산 코드를 해독하여 해당 연산을 수행하기 위한 루틴의 시작 주소를 결정
제어 주소 레지스터 (CAR : control address register)	다음에 실행할 마이크로 명령어의 주소를 저장하는 레지스터 → 이 주소는 제어 기억장치의 특정 위치를 지칭
제어 기억장치 (control memory)	마이크로 명령어들로 이루어진 마이크로 프로그램을 저장하는 내부 기억장치
제어 버퍼 레지스터 (CBR : control buffer register)	제어기억장치로부터 읽혀진 마이크로 명령어 비트들을 일시적으로 저장하는 레지스터
서브루틴 레지스터 (SBR : subroutine register)	마이크로 프로그램에서 서브루틴이 호출되는 경우에 현재의 CAR 내용을 일시적으로 저장하는 레지스터
순서제어모듈 (sequencing module)	마이크로 명령어의 실행 순서를 결정하는 회로들의 집합
해독기 (제어신호발생기)	Clock에 따라 제어신호를 발송/수신 장치

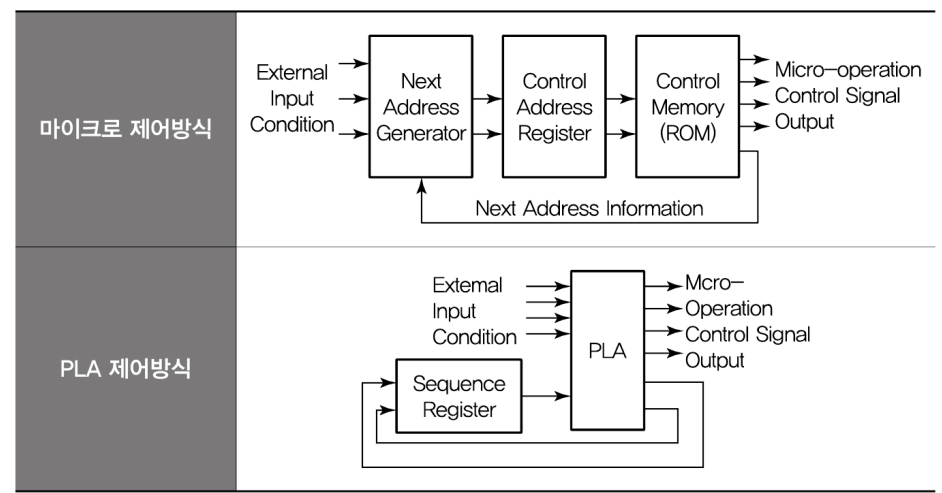
© 2021 Computer System

Hard-wired 방식(고정 하드웨어 방식)



© 2021 Computer System

마이크로 프로그래밍 방식(S/W 방식)



© 2021 Computer System

구현 방식 비교

구분	고정배선 방식	S/W 방식
속도	하드웨어 회로로 제어신호를 생성하여 고속 처리	소프트웨어적 제어신호 생성으로 상대적으로 저속
변경 유연성	하드웨어 고정회선이므로 변경이 어려움 H/W 설계 용이	Firmware 구조이므로 기능개선이 용이함, H/W 단순
소요 비용	고가(H/W 제작비용 증가)	저가(H/W 제작비용이 적게 듦)
주 적용방식	RISC	CISC
오류	오류 발생률이 높음	오류발생률이 낮음
유형	상태 F/F(병렬), 순차 레지스터와 디코더 제어(직렬)	마이크로 프로그램, PLA 제어

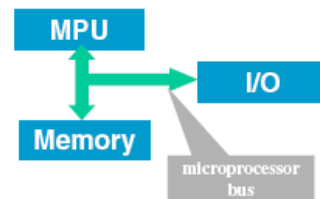
© 2021 Computer System

A Simple Processor

MU 0

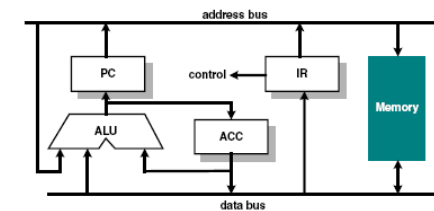
A Simple Processor

- Based on von Neumann model
- Instruction for the micro-processor (stored program) and the data for computation are both stored in **memory**
- Microprocessing Unit (MPU)** contains:
 - Arithmetic/Logic Unit (ALU)
 - Control Unit
 - Registers
- Input/output** block interfaces to the outside world (e.g. user, disc storage etc.)
- The communication between memory, MPU and I/O is through the **microprocessor bus**



© 2021 Computer System

MU0 – Very simple processor



- Let us design a simple processor MU0 with 16-bit instruction and minimal hardware:-
 - Program Counter (PC) - holds address of the next instruction to execute
 - Accumulator (ACC) - holds data being processed
 - Arithmetic Logic Unit (ALU) - performs operations on data
 - Instruction Register (IR) - holds current instruction code being executed

© 2021 Computer System

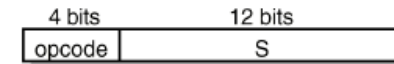
MU0

- A **Program Counter** (PC) register to hold the address of the current instruction. (Visible to Programmer)
- One register - the **Accumulator** (ACC). (Visible to Programmer).
- An **Arithmetic-Logic Unit** (ALU) for doing, eh, arithmetic and logic. (Invisible to Programmer).
- An **Instruction Register** (IR) for holding the current instruction. (Invisible to Programmer.)
- Instruction Decode Logic and Control Logic to use the components of MU0 to give effect to the instructions.

© 2021 Computer System

MU0 instructions

- Let us further assume that the processor only has 8 instructions and can only access a maximum of 8k byte of memory. This implies that the address bus is only 12-bit wide.
- We also assume that this is a 16-bit MPU and ALL instructions are 16 bits wide.
- The 16-bit instruction code (machine code) has a format:



- Note that top 4 bits define the operation code (opcode), i.e. it defines what operation the instruction is to perform.
- The bottom 12 bits define the memory address of the operand data.

© 2021 Computer System

MU0 Instruction Set

Instruction	Opcode	Semantics
LDA S	0000	$ACC := mem_{16}[S]$
STO S	0001	$mem_{16}[S] := ACC$
ADD S	0010	$ACC := ACC + mem_{16}[S]$
SUB S	0011	$ACC := ACC - mem_{16}[S]$
JMP S	0100	$PC := S$
JGE S	0101	if $ACC \geq 0$ $PC := S$
JNE S	0110	if $ACC \neq 0$ $PC := S$
STP	0111	stop

- 2 load/store instructions: LDA, STO
- 2 computation instructions: ADD, SUB
- 4 control flow instructions: JMP, JGE, JNE, STP

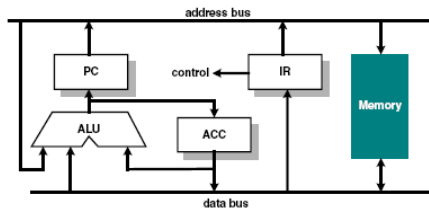
© 2021 Computer System

Data Path & Control Logic

- Data Path : all the components that carry or handle bits in parallel
 - ACC, PC, the ALU, IR
 - memory
 - Buses
- Control Logic : the logic needed to use the datapath components to give effect to the instructions in the instruction set. A finite state machine.

© 2021 Computer System

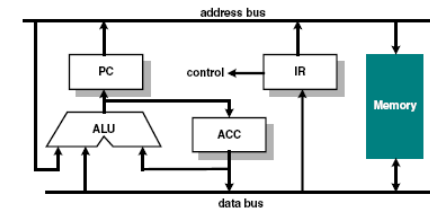
Execution Step 1 : Fetch Instruction



- MPU outputs value of program counter (PC) on address bus.
- Memory puts contents at the instruction address on the data bus.
- Instruction is stored in instruction registers (IR).

© 2021 Computer System

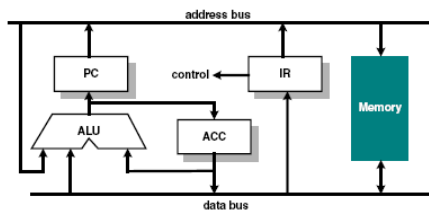
Execution Step 2 : Instruction Decode



- The instruction word stored in IR is decoded by internal logic to provide control signals to ALU and other internal circuits inside MPU.
- Program counter value (PC) is pushed onto the address bus, the ALU increment this value by **k** and put it back into the Program Counter.

© 2021 Computer System

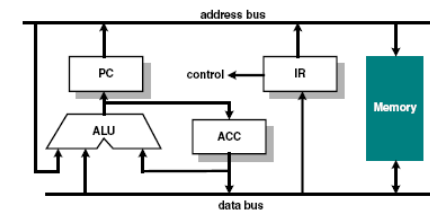
Execution Step 3 : Operand Fetch



- The instruction register provides the address of the data to be processed (i.e. **operand address**).
- Memory supplies the operand data on the data bus to the MPU, ready for processing either by the ALU or the ACC.

© 2021 Computer System

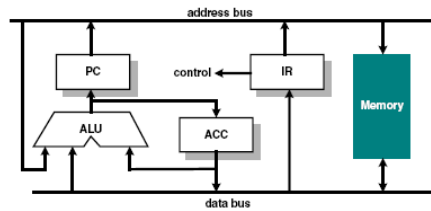
Execution Step 4 : Execute Instruction



- Processing is performed on the operand by the ALU according to the instruction.
- The result is put back into the Accumulator (ACC).

© 2021 Computer System

Execution Step 5 : Write Back (may not exist)



- This step may not be needed.
- The result from the Accumulator is written back into memory. In many processors, this will be done as a separate instruction.